

SOFTWARE MANAGEMENT SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award
of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

Shaik Abdul Sattar – 25B11AIA91

K Naveen Kumar – 25B11AI611

K Syam Kumar – 25B11AI125

K Harsha Vardhan – 25B11AI578

Under the esteemed guidance of

Mr. T.Srinivasulu M.Tech, (Ph.D.)

Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “Software Management System” is being submitted by Shaik Abdul Sattar (Roll No. 25B11AIA91), K Naveen Kumar (Roll No. 25B11AI611), K Syam Kumar (Roll No. 25B11AI125) and K Harsha Vardhan (Roll No. 25B11AI578) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the Capstone Project for the course Database Management Systems (2501IT05) during the academic year 2026–2027, under the guidance of Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem.

Project Guide

Mr. T. Srinivasulu, M. Tech, (Ph.D.)
Associate professor

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.
Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “Software Management System” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

Shaik Abdul Sattar (25B11AIA91)

K Naveen Kumar (25B11AI611)

K Syam Kumar (25B11AI125)

K Harsha Vardhan (25B11AI578)

Date: 21-09-2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr. K. Naga Bhargavi, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

Shaik Abdul Sattar	25B11AIA91
K Naveen Kumar	25B11AI611
K Syam Kumar	25B11AI125
K Harsha Vardhan	25B11AI578

ABSTRACT

The Software Management System is a database-oriented application designed to organize the information involved in planning and tracking software projects. Software teams handle project details, clients, employees, departments, tasks, milestones, teams and resources. When these records are maintained separately in spreadsheets or documents, duplication, inconsistent updates and delayed reporting can occur. The purpose of this project is to design a centralized relational database that stores and connects these records reliably. The proposed database contains eight entities represented by the Department, Employee, Client, Project, Task, Team, Milestone and Resource tables. Primary keys uniquely identify records, while foreign keys maintain relationships and prevent orphan data. The system is implemented using SQLite, with SQL used for table creation, insertion, modification, deletion and retrieval. The project demonstrates joins, aggregate functions, grouping, nested subqueries, views, set operations and transaction control. The schema is organized up to Third Normal Form to reduce redundancy and update anomalies. Sample records and validation cases are used to demonstrate the design. The result is a structured database foundation that supports project tracking, task assignment, milestone monitoring and resource organization. A full production web application, authentication and live collaboration are outside the current database prototype and are considered future enhancements.

INDEX

Chapter	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Abstract	iv
	List of Figures	v
	List of Tables	vi
1	Introduction	
	1.1 Introduction to the Domain	1
	1.2 Problem Statement	2
	1.3 Motivation and Need for a Database	3
	1.4 Objectives of the Project	4
	1.5 Scope of the Project	5
	1.6 Organization of the Report	6
2	System Study and Tools	7
	2.1 Existing System and Its Limitations	8
	2.2 Proposed Database System	9
	2.3 Tools and Technologies Used	10
	2.4 Software and Hardware Requirements	11
3	Database Design (Review – I)	12
	3.1 Identified Entities	13
	3.2 Entity Attributes and Attribute Types	14
	3.3 ER Diagram	15
	3.4 Relationships and Cardinality	16
	3.5 Relational Database Schema	17
	3.6 Primary Keys and Foreign Keys	18
	3.7 Normalization up to 3NF	19
4	Database Implementation (Review – II)	20
	4.1 Database and Table Creation – DDL	21
	4.2 Sample Data Insertion – DML	22
	4.3 Constraints and Data Integrity	23
	4.4 Data Validation Rules	24
5	SQL Operations and Queries	25
	5.1 DQL – Basic Retrieval Queries	26
	5.2 Join Operations	27
	5.3 Aggregate Functions, GROUP BY and HAVING	28
	5.4 Nested and Correlated Subqueries	29
	5.5 Views and Relational Set Operations	30
	5.6 DCL and TCL Operations	31
6	Modules and Functionalities	32
7	Results – Sample Outputs and Screenshots	33

8	Testing and Data Validation	34
9	GitHub Repository and Team Contribution	35
10	Conclusion and Future Enhancements	37
	References	40
	Appendix A – Complete SQL Script	44

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Fig. 1	ER Diagram of Software Management System	12
Fig. 2	Relational Schema Diagram	13
Fig. 3	Successful Creation of Tables	16
Fig. 4	Sample Data Inserted	17
Fig. 5	Referential Integrity Validation	18
Fig. 6	Project Retrieval Output	22
Fig. 7	Project-Client Join Output	22
Fig. 8	Aggregate Query Output	23
Fig. 9	Foreign-Key Error Output	23

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 1	Tools and Technologies Used	11
Table 2	Software and Hardware Requirements	11
Table 3	Entities Identified	12
Table 4	Attributes of Each Entity	12
Table 5	Relationships and Cardinality	13
Table 6	Relational Database Schema	13
Table 7	Primary and Foreign Key Definitions	14
Table 8	Normalization Steps	14
Table 9	Sample Data Summary	16
Table 10	Constraints and Data Integrity Rules	17
Table 11	Basic Retrieval Queries	19
Table 12	Join Operations	19
Table 13	Aggregate Queries	19
Table 14	Subqueries	20
Table 15	Views and Set Operations	20
Table 16	DCL and TCL Operations	20
Table 17	Modules and Functionalities	21
Table 18	Test Cases and Results	24
Table 19	Team Responsibilities	25
Table 20	Review-wise Progress	25

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

Software project management is the process of planning, organizing, monitoring and controlling the work required to develop and maintain software. A project typically involves a client, a project manager, employees, a team, tasks, milestones and resources. Each activity generates data that must be recorded accurately so that progress and responsibilities can be reviewed.

A Software Management System provides a structured way to maintain this information. The database in this project connects client requirements with projects, tasks, assigned employees, milestones and team resources. It helps users retrieve project-related information without searching through multiple independent files.

1.2 Problem Statement

In a manual or spreadsheet-based environment, project information may be stored in separate files by different team members. The same client or employee details can be entered repeatedly, updates may not reach every file, and task ownership or project status can become inconsistent. Searching for project progress and checking relationships is time-consuming. The proposed relational database centralizes records, uses primary and foreign keys to preserve relationships, and applies constraints to reject invalid data.

1.3 Motivation and Need for a Database

A DBMS is needed because project information is related and changes frequently. A relational database reduces repeated data, supports reliable updates, enforces key and value constraints, and allows multiple tables to be queried together. SQL also makes it possible to produce summaries such as tasks assigned to employees, milestones for a project and resources allocated to a team.

1.4 Objectives of the Project

The project objectives are:

1. To design a normalized relational database for the selected domain with at least six entities.

2. To implement the database with appropriate primary keys, foreign keys and integrity constraints.
3. 3. To organize software-management information into eight related tables with clearly defined attributes.
4. 4. To implement sample data and demonstrate CRUD operations, joins, aggregate queries, subqueries and views using SQLite.
5. 5. To validate referential integrity and document test cases, query outputs and team contributions.

1.5 Scope of the Project

The scope includes maintaining departments, employees, clients, projects, tasks, teams, milestones and resources; defining relationships among them; and retrieving operational summaries through SQL. It includes schema design, normalization, constraints, sample data and query testing. It does not include a deployed web interface, user login, real-time collaboration, automated scheduling, notifications, payroll, budgeting or integration with external project-management platforms.

1.6 Organization of the Report

This report is organized into ten chapters. Chapter 2 presents the system study and the tools used. Chapter 3 covers the complete database design including the ER diagram, relational schema and normalization. Chapter 4 describes the database implementation with DDL and sample data. Chapter 5 presents the SQL operations and queries executed. Chapter 6 explains the modules and functionalities. Chapter 7 presents the results with sample outputs. Chapter 8 covers testing and data validation. Chapter 9 details the GitHub repository and the contribution of each team member. Chapter 10 concludes the report and outlines future enhancements.

CHAPTER 2: SYSTEM STUDY AND TOOLS

2.1 Existing System and Its Limitations

The existing approach often relies on spreadsheets, text documents, email and separate tracking sheets. Its limitations include repeated data entry, inconsistent values, weak validation, difficulty tracking task ownership, slow report preparation, accidental deletion and the possibility of records referring to nonexistent projects or employees. File-based records also make it difficult to enforce relationships between clients, projects, teams and resources.

2.2 Proposed Database System

The proposed Software Management System uses an SQLite relational database with eight connected tables. Each table stores one category of information, and primary/foreign-key constraints connect the records. The system supports insert, update, delete and retrieval operations, along with joins and summary queries. This approach improves consistency, reduces redundancy and provides a foundation for future application development.

2.3 Tools and Technologies Used

Table 2.1: Tools and Technologies Used

Category	Tool / Technology Used	Version / Remarks
Database engine	SQLite	SQLite 3; lightweight relational DBMS
SQL editor / execution	SQLiteStudio or DB Browser for SQLite	Use whichever is installed; run .sql scripts
ER diagram	draw.io / diagrams.net	ER and relational-schema diagrams
Programming / scripting	SQL	DDL, DML, DQL, TCL demonstrations
Version control	GitHub	Repository: DBMS-PROJECT-PROJECT
Documentation	MS Word	Capstone report

The selected implementation stack is SQLite with SQLiteStudio or DB Browser for SQLite, draw.io for diagrams, GitHub for version control and MS Word for documentation. No separate front end is included in this database prototype.

2.4 Software and Hardware Requirements

Table 2.2: Software and Hardware Requirements

Requirement Type	Specification
Operating system	Windows 10/11, Linux or macOS

Processor	Dual-core 2.0 GHz or higher
RAM	4 GB minimum; 8 GB recommended
Storage	At least 500 MB free for tools and project files
Software	SQLite 3, SQLiteStudio/DB Browser, draw.io, MS Word, Git
Display	1366 × 768 resolution or higher; internet for GitHub

CHAPTER 3: DATABASE DESIGN

(Deliverables of Review – I)

3.1 Identified Entities

Eight entities are identified for the Software Management System. Each entity represents one major data category and is implemented as a separate relational table.

Table 3.1: Entities Identified for the Selected Domain

S.No	Entity Name	Description
1	Department	Stores organizational departments.
2	Employee	Stores employee details and department membership.
3	Client	Stores client/customer information.
4	Project	Stores software project details for a client.
5	Task	Stores project tasks and assigned employees.
6	Team	Stores a project team and its team lead.
7	Milestone	Stores dated milestones for a project.
8	Resource	Stores resources assigned to a team.

3.2 Entity Attributes and Attribute Types

The attribute catalogue below lists the fields, SQLite data types, key roles and attribute classifications for all eight entities. All listed attributes are simple, single-valued attributes.

Table 3.2: Attributes of Each Entity

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Department	department id	INTEGER	PK	Simple
Department	department name	TEXT	—	Simple
Department	location	TEXT	—	Simple
Employee	employee id	INTEGER	PK	Simple
Employee	first name	TEXT	—	Simple
Employee	last name	TEXT	—	Simple
Employee	email	TEXT	—	Simple
Employee	job title	TEXT	—	Simple
Employee	hire date	TEXT	—	Simple
Employee	department id	INTEGER	FK	Simple
Client	client id	INTEGER	PK	Simple
Client	client name	TEXT	—	Simple
Client	contact email	TEXT	—	Simple
Client	phone	TEXT	—	Simple
Client	address	TEXT	—	Simple
Project	project id	INTEGER	PK	Simple
Project	project name	TEXT	—	Simple
Project	description	TEXT	—	Simple
Project	start date	TEXT	—	Simple
Project	end date	TEXT	—	Simple
Project	status	TEXT	—	Simple
Project	client id	INTEGER	FK	Simple
Task	task id	INTEGER	PK	Simple
Task	task name	TEXT	—	Simple
Task	description	TEXT	—	Simple
Task	priority	TEXT	—	Simple
Task	status	TEXT	—	Simple

Task	due date	TEXT	—	Simple
Task	project id	INTEGER	FK	Simple
Task	employee id	INTEGER	FK	Simple
Team	team id	INTEGER	PK	Simple
Team	team name	TEXT	—	Simple
Team	team lead id	INTEGER	FK	Simple
Team	project id	INTEGER	FK, UNIQUE	Simple
Milestone	milestone id	INTEGER	PK	Simple
Milestone	milestone name	TEXT	—	Simple
Milestone	due date	TEXT	—	Simple
Milestone	status	TEXT	—	Simple
Milestone	project id	INTEGER	FK	Simple
Resource	resource id	INTEGER	PK	Simple
Resource	resource name	TEXT	—	Simple
Resource	resource type	TEXT	—	Simple
Resource	quantity	INTEGER	—	Simple
Resource	team id	INTEGER	FK	Simple

3.3 ER Diagram

The ER diagram presents the eight entities and their principal relationships. Primary keys identify each entity; foreign-key relationships connect the records. The diagram is included below.

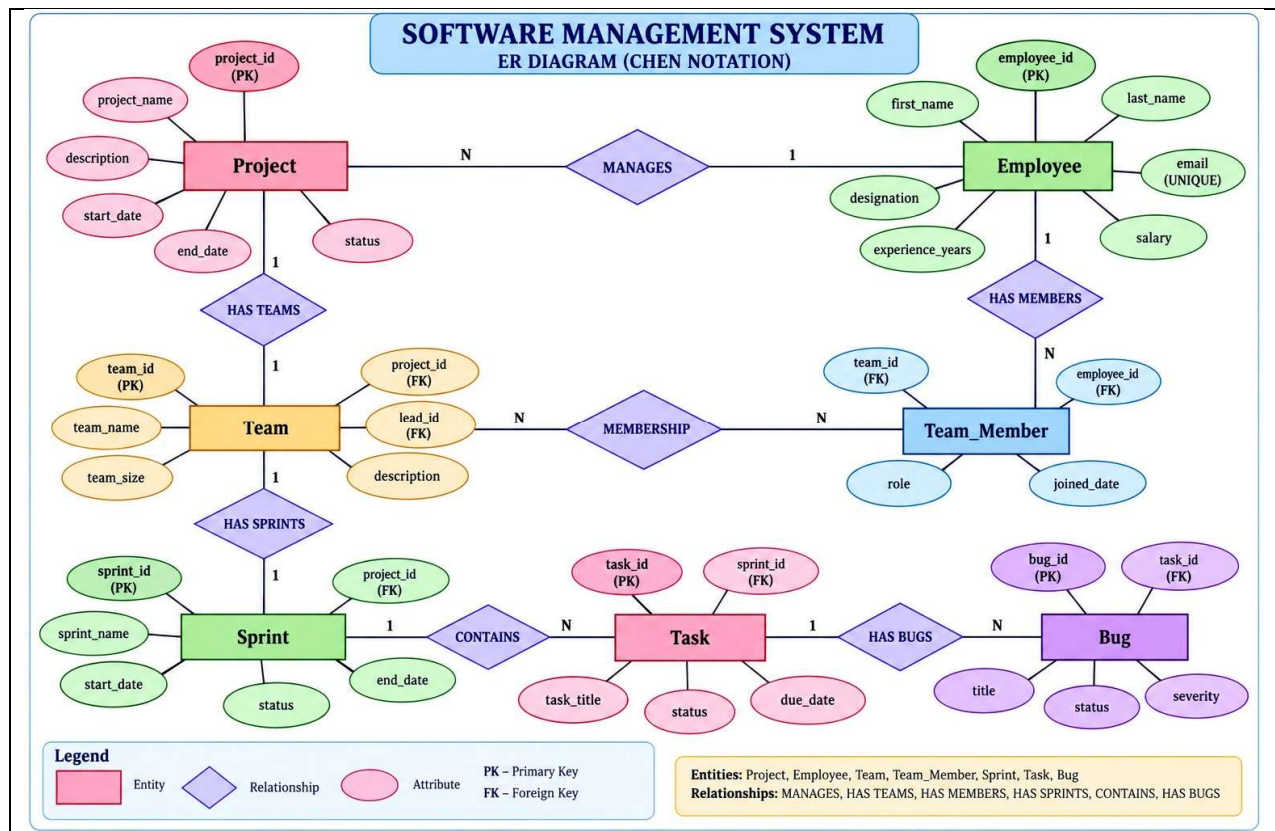


Fig. 3.1: ER Diagram of the Software Management System

3.4 Relationships and Cardinality

Table 3.3: Relationships and Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)	Participation
Project	MANAGES	Employee	N:1	Not specified in diagram
Project	HAS TEAMS	Team	1:1	Not specified in diagram
Employee	HAS MEMBERS	Team_Member	1:N	Not specified in diagram
Team	MEMBERSHIP	Team_Member	N:N	Not specified in diagram
Team	HAS SPRINTS	Sprint	1:1	Not specified in diagram
Sprint	CONTAINS	Task	1:N	Not specified in diagram
Task	HAS BUGS	Bug	1:N	Not specified in diagram

3.5 Relational Database Schema

The relational schema maps each identified entity to a table. The following catalogue records every column, its SQLite data type, key designation and principal constraint.

Table 3.4: Relational Database Schema

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Department	department_id	INTEGER	PK	PRIMARY KEY
Department	department_name	TEXT	—	NOT NULL, UNIQUE
Department	location	TEXT	—	
Employee	employee_id	INTEGER	PK	PRIMARY KEY
Employee	first_name	TEXT	—	
Employee	last_name	TEXT	—	
Employee	email	TEXT	—	NOT NULL, UNIQUE
Employee	job_title	TEXT	—	
Employee	hire_date	TEXT	—	
Employee	department_id	INTEGER	FK	FOREIGN KEY
Client	client_id	INTEGER	PK	PRIMARY KEY
Client	client_name	TEXT	—	NOT NULL
Client	contact_email	TEXT	—	
Client	phone	TEXT	—	
Client	address	TEXT	—	
Project	project_id	INTEGER	PK	PRIMARY KEY
Project	project_name	TEXT	—	
Project	description	TEXT	—	
Project	start_date	TEXT	—	

Project	end_date	TEXT	—	
Project	status	TEXT	—	CHECK status IN ('Planned','Active','On Hold','Completed')
Project	client_id	INTEGER	FK	FOREIGN KEY
Task	task_id	INTEGER	PK	PRIMARY KEY
Task	task_name	TEXT	—	
Task	description	TEXT	—	
Task	priority	TEXT	—	CHECK priority IN ('Low','Medium','High')
Task	status	TEXT	—	CHECK status IN ('To Do','In Progress','Done')
Task	due_date	TEXT	—	
Task	project_id	INTEGER	FK	FOREIGN KEY
Task	employee_id	INTEGER	FK	FOREIGN KEY
Team	team_id	INTEGER	PK	PRIMARY KEY
Team	team_name	TEXT	—	
Team	team_lead_id	INTEGER	FK	FOREIGN KEY
Team	project_id	INTEGER	FK, UNIQUE	FOREIGN KEY, UNIQUE; 1:1 project link
Milestone	milestone_id	INTEGER	PK	PRIMARY KEY
Milestone	milestone_name	TEXT	—	
Milestone	due_date	TEXT	—	
Milestone	status	TEXT	—	
Milestone	project_id	INTEGER	FK	FOREIGN KEY
Resource	resource_id	INTEGER	PK	PRIMARY KEY
Resource	resource_name	TEXT	—	
Resource	resource_type	TEXT	—	
Resource	quantity	INTEGER	—	CHECK quantity >= 0
Resource	team_id	INTEGER	FK	FOREIGN KEY

Schema Diagram

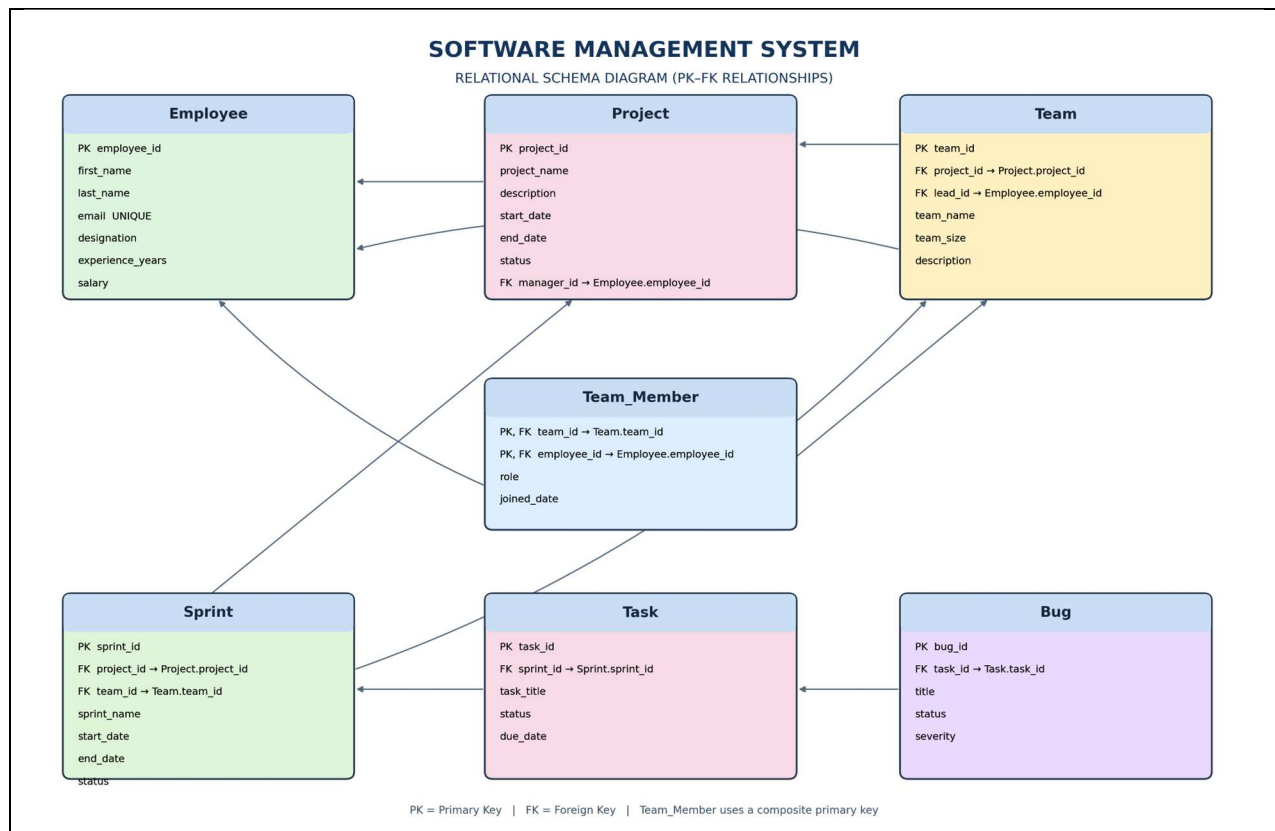


Fig. 3.2: Relational Schema Diagram

3.6 Primary Keys and Foreign Keys

Table 3.5: Primary Key and Foreign Key Definitions

Table	Primary Key	Foreign Key	References (Table.Column)
Department	department_id	—	—
Employee	employee_id	department_id	Department.department_id
Client	client_id	—	—
Project	project_id	client_id	Client.client_id
Task	task_id	project_id; employee_id	Project.project_id; Employee.employee_id
Team	team_id	team_lead_id; project_id	Employee.employee_id; Project.project_id
Milestone	milestone_id	project_id	Project.project_id
Resource	resource_id	team_id	Team.team_id

3.7 Normalization up to 3NF

The initial unnormalized project register may contain repeated employee, client, task, milestone and resource details in one large record. Functional dependencies include: department_id → department_name, location; employee_id → employee details, department_id; client_id → client details; project_id → project details, client_id; task_id → task details, project_id, employee_id; team_id → team details, project_id; milestone_id → milestone details, project_id; resource_id →

resource details, team_id. The decomposition below removes repeating groups, partial dependencies and transitive dependencies.

UNF → 1NF → 2NF → 3NF

Table 3.6: Normalization Steps

Normal Form	Condition Satisfied	Anomaly Removed	Resulting Tables
UNF	One project record may contain repeating task/resource/milestone lists.	Difficult inserts; repeated values and inconsistent updates.	Single unnormalized project register
1NF	Each field contains one atomic value; repeating groups are separated.	Repeating groups and non-atomic values.	Department, Employee, Client, Project, Task, Team, Milestone, Resource records
2NF	Each non-key attribute depends on the whole primary key; each table uses a single-column PK.	Partial dependency on part of a composite key.	Eight entity tables with individual primary keys
3NF	Non-key attributes depend only on the key, not on other non-key attributes. Department/client/project/team details are separated.	Transitive dependency and update anomalies.	Final 8-table schema; FKs link related entities

CHAPTER 4: DATABASE IMPLEMENTATION

(Deliverables of Review – II)

4.1 Database and Table Creation – DDL

The database is implemented in SQLite. Run the following DDL in SQLiteStudio or DB Browser for SQLite. SQLite creates the database file when it is opened; therefore, CREATE DATABASE is not used. Enable foreign-key enforcement for each connection with PRAGMA foreign_keys = ON. The complete DDL and sample inserts are provided in Appendix A.

SQLite DDL: PRAGMA foreign_keys = ON; CREATE TABLE statements for Department, Employee, Client, Project, Task, Team, Milestone and Resource are included in Appendix A. Execute the script in a new SQLite database file.

Execution evidence to capture: run the Appendix A DDL in SQLiteStudio/DB Browser and take a screenshot showing successful creation of all eight tables. No execution screenshot is fabricated in this report.

Execution evidence: insert a genuine screenshot from SQLiteStudio or DB Browser for SQLite showing successful creation of all eight tables. Do not use a mock screenshot.

4.2 Sample Data Insertion – DML

Sample records are inserted into all eight tables in dependency order: Department and Client first, followed by Employee and Project, then Task, Team, Milestone and Resource. Appendix A contains the INSERT statements and examples of UPDATE and DELETE. Foreign-key enforcement should be enabled before inserting records.

Table 4.1: Sample Data Summary

S.No	Table Name	No. of Records Inserted	Screenshot Reference
1	Department	3	Fig. 4.2 / run output
2	Employee	5	Fig. 4.2 / run output
3	Client	3	Fig. 4.2 / run output
4	Project	4	Fig. 4.2 / run output
5	Task	6	Fig. 4.2 / run output
6	Team	4	Fig. 4.2 / run output
7	Milestone	5	Fig. 4.2 / run output
8	Resource	6	Fig. 4.2 / run output

Sample data script: see Appendix A. Expected record totals after initial inserts — Department 3; Employee 5; Client 3; Project 4; Task 6; Team 4; Milestone 5; Resource 6. Capture the SELECT COUNT(*) output from your SQLite tool for the final submission.

Fig. 4.2: Sample Data Inserted into the Software Management System

4.3 Constraints and Data Integrity

Table 4.2: Constraints and Data Integrity Rules

Constraint / Validation	Applied On (Table.Column)	Purpose	Evidence / Screenshot Ref.
PRIMARY KEY	All table ID columns	Prevents duplicate or NULL identifiers.	DDL run
FOREIGN KEY	Employee.department_id	Employee must reference an existing department.	FK test
FOREIGN KEY	Project.client_id	Project must reference an existing client.	FK test
FOREIGN KEY	Task.project_id, Task.employee_id	Task must reference valid project and employee.	FK test
FOREIGN KEY	Team.team_lead_id, Team.project_id	Team lead and project must exist; project link is unique.	FK/UNIQUE test
FOREIGN KEY	Milestone.project_id; Resource.team_id	Prevents orphan milestones/resources.	FK test
UNIQUE	Department.department_name; Employee.email; Team.project_id	Prevents duplicate department names, emails and project-team links.	UNIQUE test
CHECK	Project.status; Task.priority/status; Resource.quantity	Restricts values to valid statuses/priorities and non-negative quantity.	CHECK test
NOT NULL	Required names, IDs and key fields	Rejects missing mandatory values.	NOT NULL test

4.4 Data Validation Rules

Primary keys prevent duplicate identifiers. UNIQUE constraints prevent duplicate department names, employee emails and project-to-team assignments. CHECK constraints restrict project/task statuses, task priority and resource quantity. NOT NULL constraints require essential values. Foreign keys prevent orphan records when PRAGMA foreign_keys = ON. Example violation: INSERT INTO Task (task_id, task_name, priority, status, due_date, project_id, employee_id) VALUES (999, 'Invalid FK test', 'High', 'To Do', '2026-12-01', 9999, 1); This should fail with a FOREIGN KEY constraint error because project_id 9999 does not exist. Exact error wording depends on the SQLite client.

Expected validation result: FOREIGN KEY constraint failed. Run the invalid Task INSERT from Section 4.4 with PRAGMA foreign_keys = ON, then capture the actual error message from SQLite for the final report.
--

Fig. 4.3: Referential Integrity Enforced by the Database (capture actual SQLite error)

CHAPTER 5: SQL OPERATIONS AND QUERIES

The following query catalogue covers retrieval, joins, aggregates, subqueries, views, set operations and transaction/security commands. Execute the statements against the Appendix A sample database. Output references identify the query/result to capture; outputs depend on the final sample data loaded.

5.1 DQL – Basic Retrieval Queries

Table 5.1: Basic Retrieval Queries

Q.No	Purpose of the Query	SQL Statement	Output Ref.
Q1	List all projects	SELECT project_id, project_name, status FROM Project;	Output Q1
Q2	Find active projects	SELECT * FROM Project WHERE status = 'Active';	Output Q2
Q3	List employees alphabetically	SELECT employee_id, first_name, last_name FROM Employee ORDER BY last_name;	Output Q3
Q4	Find high-priority tasks	SELECT task_name, priority, status FROM Task WHERE priority = 'High';	Output Q4
Q5	Projects starting after a date	SELECT project_name, start_date FROM Project WHERE start_date >= '2026-01-01';	Output Q5

5.2 Join Operations

The join queries below demonstrate natural, equi, inner, left outer, right outer (where supported, or its equivalent) and full outer joins across related Software Management System tables.

Table 5.2: Join Operations

Q.No	Type of Join	SQL Statement	Output Ref.
Q6	Natural Join	SELECT department_name, first_name, last_name FROM Employee NATURAL JOIN Department;	Output Q6
Q7	Equi Join	SELECT e.first_name, d.department_name FROM Employee e, Department d WHERE e.department_id = d.department_id;	Output Q7
Q8	Inner Join	SELECT p.project_name, c.client_name FROM Project p INNER JOIN Client c ON p.client_id=c.client_id;	Output Q8
Q9	Left Outer Join	SELECT c.client_name, p.project_name FROM Client c LEFT JOIN Project p ON c.client_id=p.client_id;	Output Q9
Q10	Right Outer Join	SELECT p.project_name, c.client_name FROM Project p LEFT JOIN Client c ON p.client_id=c.client_id; -- SQLite equivalent preserving all projects	Output Q10
Q11	Full Outer Join	SELECT c.client_name, p.project_name FROM Client c FULL OUTER JOIN Project p ON c.client_id=p.client_id; -- SQLite 3.39+	Output Q11

5.3 Aggregate Functions, GROUP BY and HAVING

Table 5.3: Aggregate, GROUP BY and HAVING Queries

Q.No	Function Used	SQL Statement	Output Ref.
Q12	COUNT / GROUP BY	SELECT status, COUNT(*) AS project_count FROM Project GROUP BY status;	Output Q12
Q13	AVG / GROUP BY	SELECT project_id, AVG(CASE priority WHEN 'High' THEN 3 WHEN 'Medium' THEN 2 ELSE 1 END) AS avg_priority_score FROM Task GROUP BY project_id;	Output Q13
Q14	COUNT / HAVING	SELECT project_id, COUNT(*) AS task_count FROM Task GROUP BY project_id HAVING COUNT(*) >= 2;	Output Q14
Q15	MIN / MAX	SELECT MIN(due_date) AS earliest_due, MAX(due_date) AS latest_due FROM Task;	Output Q15
Q16	SUM	SELECT team_id, SUM(quantity) AS total_resource_units FROM Resource GROUP BY team_id;	Output Q16

5.4 Nested and Correlated Subqueries

Table 5.4: Subqueries

Q.No	Type (Nested / Correlated)	SQL Statement	Output Ref.
Q17	Nested	SELECT project_name FROM Project WHERE client_id IN (SELECT client_id FROM Client WHERE client_name LIKE '%Tech%');	Output Q17
Q18	Nested	SELECT task_name FROM Task WHERE employee_id IN (SELECT employee_id FROM Employee WHERE department_id = 1);	Output Q18
Q19	Correlated	SELECT e.employee_id, e.first_name FROM Employee e WHERE EXISTS (SELECT 1 FROM Task t WHERE t.employee_id=e.employee_id);	Output Q19
Q20	Correlated	SELECT p.project_name FROM Project p WHERE (SELECT COUNT(*) FROM Task t WHERE t.project_id=p.project_id) > 1;	Output Q20
Q21	Nested	SELECT milestone_name FROM Milestone WHERE project_id = (SELECT project_id FROM Project WHERE project_name = 'Website Redesign' LIMIT 1);	Output Q21

5.5 Views and Relational Set Operations

The following examples include a simple single-table view, a grouped summary view, and UNION, INTERSECT and EXCEPT set operations. SQLite supports EXCEPT rather than MINUS.

Table 5.5: Views and Set Operations

Q.No	View / Set Operation	SQL Statement	Output Ref.
Q22	Updatable View	CREATE VIEW ActiveProjects AS SELECT project_id, project_name, status, client_id FROM Project WHERE status='Active'; -- simple single-table view	View Q22
Q23	Non-updatable View	CREATE VIEW ProjectTaskSummary AS SELECT p.project_name, COUNT(t.task_id) AS task_count FROM Project p LEFT JOIN Task t ON p.project_id=t.project_id GROUP BY p.project_id;	View Q23
Q24	UNION	SELECT email AS contact FROM Employee UNION SELECT contact_email FROM Client;	Output Q24
Q25	INTERSECT	SELECT email FROM Employee INTERSECT SELECT contact_email FROM Client;	Output Q25
Q26	EXCEPT (SQLite MINUS equivalent)	SELECT email FROM Employee EXCEPT SELECT contact_email FROM Client;	Output Q26

5.6 DCL and TCL Operations

DCL examples are shown for DBMS platforms that support GRANT and REVOKE; SQLite does not implement these commands. TCL examples demonstrate transaction control using COMMIT, SAVEPOINT and ROLLBACK.

Table 5.6: DCL and TCL Operations

Q.No	Command	Purpose	SQL Statement	Output Ref.
Q27	GRANT	Role-based permission example (not supported by SQLite SQL engine)	GRANT SELECT ON Project TO analyst; -- conceptual; use DBMS with GRANT support	Not executable in SQLite
Q28	REVOKE	Revoke role permission (not supported by SQLite SQL engine)	REVOKE SELECT ON Project FROM analyst; -- conceptual; use DBMS with REVOKE support	Not executable in SQLite
Q29	COMMIT	Make transaction changes permanent	BEGIN TRANSACTION; UPDATE Task SET status='In Progress' WHERE task_id=1;	Transaction output

			COMMIT;	
Q30	SAVEPOINT	Mark a point within a transaction	BEGIN; SAVEPOINT before_update; UPDATE Task SET status='Done' WHERE task_id=1;	Transaction output
Q31	ROLLBACK TO	Undo changes after savepoint	ROLLBACK TO before_update; RELEASE before_update; ROLLBACK;	Transaction output
Q32	ROLLBACK	Cancel uncommitted transaction changes	BEGIN; DELETE FROM Resource WHERE resource_id=6; ROLLBACK;	Transaction output

CHAPTER 6: MODULES AND FUNCTIONALITIES

This chapter describes the functional modules represented by the database tables and the SQL operations used to maintain and retrieve software-management information.

Table 6.1: Modules and Their Functionalities

S.No	Module Name	Functionality	Tables Involved
1	Department & Employee Management	Maintain departments and employee profiles; connect employees to departments.	Department, Employee
2	Client Management	Store client contact details and connect clients to projects.	Client, Project
3	Project Management	Create projects, track dates/status and associate each project with a client.	Project, Client, Team
4	Task Assignment & Tracking	Record task priority, due date, status, project and assigned employee.	Task, Project, Employee
5	Team Management	Maintain project team and identify team lead; enforce one team per project.	Team, Employee, Project
6	Milestone Tracking	Record project milestones, due dates and completion status.	Milestone, Project
7	Resource Management	Track resource name, type and quantity assigned to each team.	Resource, Team
8	Reporting & Query Module	Retrieve project/task details and produce counts, joins and summaries.	All tables

The Department & Employee module maintains organizational structure and employee details. Client and Project Management connect customer records with the software projects they sponsor. Task Assignment records work items, priorities, due dates, statuses and responsible employees. Team Management associates a team and its lead with a project. Milestone Tracking stores important project dates and status, while Resource Management records resources allocated to teams. The Reporting module uses SQL joins, filters and aggregate functions to present combined information across the database.

CHAPTER 7: RESULTS – SAMPLE OUTPUTS AND SCREENSHOTS

The following result panels identify four key outputs to capture from the SQLite client. Replace each capture instruction with a screenshot showing both the executed SQL statement and its actual result before final submission.

Sample output to capture: SELECT project_id, project_name, status FROM Project;
Expected columns: project_id | project_name | status
Run Q1 in SQLite and paste the actual query + result screenshot here.

Fig. 7.1: Project records retrieved using Q1

Sample output to capture: SELECT p.project_name, c.client_name FROM Project p JOIN Client c ON p.client_id=c.client_id;
Expected columns: project_name | client_name
Run Q8 in SQLite and paste the actual query + result screenshot here.

Fig. 7.2: Project and client details using an INNER JOIN (Q8)

Sample output to capture: SELECT status, COUNT(*) FROM Project GROUP BY status;
Expected columns: status | COUNT(*)
Run Q12 in SQLite and paste the actual query + result screenshot here.

Fig. 7.3: Project count grouped by status (Q12)

Sample output to capture: invalid Task INSERT referencing project_id = 9999 with foreign keys enabled.
Expected: FOREIGN KEY constraint failed
Paste the actual error screenshot here.

Fig. 7.4: Foreign-key validation error for an invalid project reference

CHAPTER 8: TESTING AND DATA VALIDATION

The test cases below cover valid inserts, duplicate keys, invalid values, foreign-key violations and query correctness. Record the observed result after executing each test against the final database.

Table 8.1: Test Cases and Results

TC No.	Test Case Description	Input / Action	Expected Result	Actual Result	Status
TC01	Insert valid department	Insert unique department name	Row inserted	To be recorded after execution	Pending
TC02	Duplicate primary key	Insert existing department_id	PRIMARY KEY constraint error	To be recorded after execution	Pending
TC03	Duplicate unique email	Insert employee with existing email	UNIQUE constraint error	To be recorded after execution	Pending
TC04	Invalid project status	Insert status = DraftX	CHECK constraint error	To be recorded after execution	Pending
TC05	Negative resource quantity	Insert quantity = -1	CHECK constraint error	To be recorded after execution	Pending
TC06	Orphan employee department	Insert invalid department_id with FK ON	FOREIGN KEY constraint error	To be recorded after execution	Pending
TC07	Orphan task project	Insert invalid project_id with FK ON	FOREIGN KEY constraint error	To be recorded after execution	Pending
TC08	Valid join query	Execute Q8	Project and matching client displayed	To be recorded after execution	Pending
TC09	Aggregate correctness	Execute Q12	Counts match rows grouped by status	To be recorded after execution	Pending

CHAPTER 9: GITHUB REPOSITORY AND TEAM CONTRIBUTION

9.1 Repository Details

GitHub Repository Link: <https://github.com/> (repository name: DBMS-PROJECT-PROJECT; insert the exact repository URL after confirming it)

The repository should contain the final SQL script, project documentation, ER diagram, relational schema diagram and supporting evidence. Replace the repository placeholder with the confirmed team-repository URL before submission.

9.2 Team Responsibilities and Contribution

Table 9.1: Team Responsibilities

S.No	Student Name	Roll Number	Role / Responsibility	Modules Contributed
1	Shaik Abdul Sattar	25B11AIA91	Team coordination, schema documentation and query integration	Project, SQL queries, report
2	K Naveen Kumar	25B11AI611	Database table design and sample data preparation	Department, Employee, Client
3	K Syam Kumar	25B11AI125	ER diagram, relationships and normalization	ER/schema diagrams, normalization
4	K Harsha Vardhan	25B11AI578	Testing, validation and documentation support	Constraints, testing, screenshots

9.3 Review-wise Progress

Table 9.2: Review-wise Progress Record

Review	Date	Deliverables Presented	Remarks of the Guide
Review I	To be entered	Entity identification, attributes, ER diagram, schema and normalization	To be completed by guide
Review II	To be entered	DDL, sample data, constraints and SQL query demonstrations	To be completed by guide
Final Review	To be entered	Complete report, testing evidence, GitHub repository and conclusion	To be completed by guide

CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Software Management System database provides a structured solution for storing and connecting software project information. The design contains eight entities and eight relational tables: Department, Employee, Client, Project, Task, Team, Milestone and Resource. Primary and foreign keys establish entity identity and relationships, while UNIQUE, NOT NULL and CHECK constraints support data integrity. The schema is decomposed to Third Normal Form to reduce repeating data and transitive dependencies. The implementation plan includes DDL and DML, retrieval queries, joins, aggregate functions, nested/correlated subqueries, views, set operations and transaction-control examples. These deliverables address the project objectives by defining a normalized schema, implementing constraints, loading sample data and preparing validation/query tests. Actual execution screenshots and test outcomes should be added after running the SQL script in SQLite.

10.2 Limitations of the Present Work

The current work is a database prototype rather than a complete deployed application. It does not provide a graphical user interface, authentication, fine-grained user permissions in SQLite, real-time collaboration, automated notifications, dependency scheduling, time-sheet tracking, billing or integration with external tools. The test outcomes and screenshots must be confirmed against the final database instance. SQLite does not implement SQL GRANT and REVOKE, so those commands are documented as conceptual examples rather than executable SQLite statements.

10.3 Future Enhancements

Future enhancements may include a web or desktop front end, secure login with role-based access control in an application layer, stored procedures/triggers where supported, indexes for frequently searched columns, automated backups and recovery, audit logs, task dependencies, progress dashboards, notifications, resource utilization reports and integration with GitHub or other development tools. The database may also be migrated to PostgreSQL or MySQL if multi-user concurrency and server-side permissions become necessary.

REFERENCES

References used for database concepts and SQLite implementation:

6. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, “Database System Concepts”, 8th Edition, McGraw-Hill, ISBN: 978-1260230508.
7. Raghurama Krishnan and Johannes Gehrke, “Database Management Systems”, 4th Edition, McGraw-Hill, ISBN: 978-1260091195.
8. Ramez Elmasri and Shamkant B. Navathe, “Fundamentals of Database Systems”, 7th Edition, Pearson, ISBN: 978-0137504277.
9. SQLite Documentation, “CREATE TABLE” and constraint documentation,
https://www.sqlite.org/lang_createtable.html
10. SQLite Documentation, “SQL Language” and foreign key support,
<https://www.sqlite.org/lang.html> and <https://www.sqlite.org/foreignkeys.html>

APPENDIX A – COMPLETE SQL SCRIPT

The complete SQLite DDL, sample INSERT statements and representative data are included below. Add the final executed query set to the same script or to a companion queries.sql file in the team repository.

```
-- SOFTWARE MANAGEMENT SYSTEM
-- RELATIONAL DATABASE SCHEMA (SQLite)
```

```
PRAGMA foreign_keys = ON;
```

```
-- 1. DEPARTMENT TABLE
```

```
CREATE TABLE Department (
    department_id INTEGER PRIMARY KEY,
    department_name TEXT NOT NULL UNIQUE,
    location TEXT
);
```

```
-- 2. EMPLOYEE TABLE
```

```
CREATE TABLE Employee (
    employee_id INTEGER PRIMARY KEY,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    job_title TEXT,
    hire_date TEXT,
    department_id INTEGER NOT NULL,

    FOREIGN KEY (department_id)
        REFERENCES Department(department_id)
);
```

```
-- 3. CLIENT TABLE
```

```
CREATE TABLE Client (
    client_id INTEGER PRIMARY KEY,
    client_name TEXT NOT NULL,
    contact_email TEXT,
    phone TEXT,
    address TEXT
);
```

```
-- 4. PROJECT TABLE
```

```

CREATE TABLE Project (
    project_id INTEGER PRIMARY KEY,
    project_name TEXT NOT NULL,
    description TEXT,
    start_date TEXT,
    end_date TEXT,
    status TEXT NOT NULL DEFAULT 'Planned'
    CHECK (status IN ('Planned', 'Active', 'On Hold', 'Completed')),
    client_id INTEGER NOT NULL,

    FOREIGN KEY (client_id)
        REFERENCES Client(client_id)
);

```

-- 5. TASK TABLE

```

CREATE TABLE Task (
    task_id INTEGER PRIMARY KEY,
    task_name TEXT NOT NULL,
    description TEXT,
    priority TEXT NOT NULL
    CHECK (priority IN ('Low', 'Medium', 'High')),
    status TEXT NOT NULL DEFAULT 'To Do'
    CHECK (status IN ('To Do', 'In Progress', 'Done')),
    due_date TEXT,
    project_id INTEGER NOT NULL,
    employee_id INTEGER NOT NULL,

    FOREIGN KEY (project_id)
        REFERENCES Project(project_id),

    FOREIGN KEY (employee_id)
        REFERENCES Employee(employee_id)
);

```

-- 6. TEAM TABLE

```

CREATE TABLE Team (
    team_id INTEGER PRIMARY KEY,
    team_name TEXT NOT NULL,
    team_lead_id INTEGER NOT NULL,
    project_id INTEGER NOT NULL UNIQUE,

```

```
FOREIGN KEY (team_lead_id)
REFERENCES Employee(employee_id),
```

```
FOREIGN KEY (project_id)
REFERENCES Project(project_id)
```

```
);
```

```
-- 7. MILESTONE TABLE
```

```
CREATE TABLE Milestone (
    milestone_id INTEGER PRIMARY KEY,
    milestone_name TEXT NOT NULL,
    due_date TEXT,
    status TEXT NOT NULL DEFAULT 'Pending'
    CHECK (status IN ('Pending', 'Completed', 'Delayed')),
    project_id INTEGER NOT NULL,
```

```
FOREIGN KEY (project_id)
REFERENCES Project(project_id)
```

```
);
```

```
-- 8. RESOURCE TABLE
```

```
CREATE TABLE Resource (
    resource_id INTEGER PRIMARY KEY,
    resource_name TEXT NOT NULL,
    resource_type TEXT,
    quantity INTEGER NOT NULL DEFAULT 1
    CHECK (quantity >= 0),
    team_id INTEGER NOT NULL,
```

```
FOREIGN KEY (team_id)
REFERENCES Team(team_id)
```

```
);
```